

- ii) Para implementar las operaciones del TAD de manera eficiente se utilizará la estructura de datos *Pairing Heaps*. Un *pairing heap* es un heap vacío o un par formado por un elemento y una lista de pairing heaps. El invariante que debe cumplir es que el elemento de la raíz es menor o igual a todos los elementos de sus hijos.

Para implementar *pairing heap* en Haskell definimos el siguiente tipo de datos:

```
data PHeaps a = Empty | Root a [PHeaps a]
```

y lo utilizamos para definir el siguiente tipo de datos para multimaps:

```
type MultiMap a b = PHeaps (a, [b])
```

Implementar las operaciones *minKey* y *values* del TAD, definiendo en Haskell las funciones:

```
values :: Ord a => a -> MultiMap a b -> [b]
minKey :: Ord a => MultiMap a b -> [b]
```

La función *minKey* debe ser de orden $O(1)$.

3. Sea $g = (V, E)$ un grafo simple, un lado $(v, w) \in E$ es llamado **punto** si (v, w) no está incluido en ningún ciclo de g , es decir que al eliminarlo del grafo no existe más un camino de v a w .

Suponiendo que los grafos están representados como tablas de adyacencias, es decir que $Graph\ V = Table\ V\ \mathbb{S}_V$, y que se cuenta con una definición de la función $N_G : Graph\ V \rightarrow \mathbb{S}_V \rightarrow \mathbb{S}_V$ que calcula el conjunto de vecinos para un conjunto de vértices en un grafo (es decir, $N_G(S) = \cup_{v \in S} (getNbrs\ G\ v)$), definir de manera eficiente las siguientes funciones:

- $path : Graph\ v \rightarrow (v, v) \rightarrow Bool$, que determine si existe un camino entre dos vértices de un grafo.
- $elimEdge : Graph\ v \rightarrow (v, v) \rightarrow Graph\ v$, que dados un grafo, un lado del mismo, elimine la arista del grafo.
- $bridge : Graph\ v \rightarrow (v, v) \rightarrow Bool$, que dados un grafo y una arista del mismo determine si la arista es un punto del grafo.

Nota: El puntaje máximo para cada ejercicio será dado si el costo de la implementación dada coincide con el menor costo posible.